Authentication Mechanisms and Protocols

Authentication: What is Proven?

- **Authentication verifies a claim made by a claimant.**
  - A verifier checks evidence produced using an authenticator or credential.

Example: "I control the account alice@example.org."

  - A password, passkey, smart card or other authenticator provides evidence.

- **A different claim is "I am over 18".**
  - A trusted issuer can attest to an attribute; the presenter may also need to prove control of the credential.

➢ **Distinguish authentication, identity proofing, and authorization.**

Authentication Factors and MFA

- **Something you know**
  - Password or PIN
- **Something you have**
  - Security key, smartphone or smart card
- **Something you are**
  - Fingerprint, face or other biometric characteristic

- **Multi-factor authentication (MFA)**
  - Combines evidence from different factor categories.
  - Two passwords are not two factors.
  - A passkey unlocked by a PIN or biometric can provide two factors.
- **Adaptive / step-up authentication**
  - Additional verification when risk increases.
  - MFA reduces risk, but not every MFA method resists phishing.

Authentication versus Authorization

- **Authentication (Authn): Who or what is interacting?**
  - Establishes control of an account, key or authenticator.
- **Authorization (Authz): What may that entity do?**
  - Evaluates permissions, roles, attributes and context.

Example: Alice signs in (Authn); Alice may read file X (Authz).

- Authenticating an entity does not automatically grant access.

Authentication also supports secure session and key establishment.

Requirements for Secure Authentication

- **Assurance**
  - Confidence that the claimant controls an authenticator bound to the account.
- **Protection of secrets and keys**
  - Do not leak passwords, signing keys, OTP seeds or session credentials.
- **Protocol security**
  - Resist guessing, replay, interception, impersonation and phishing.
- **Operational security**
  - Support safe enrolment, recovery, revocation, logging and reauthentication.

Authentication Assurance Levels (NIST SP 800-63B-4)

AAL1

Single-factor or MFA.
Basic protection and secure channel.
No phishing resistance requirement.

AAL2

Two distinct factors.
Replay resistance required.
A phishing-resistant option must be offered.

AAL3

Phishing-resistant public-key authentication.
Non-exportable, hardware-protected key; two factors.

IAL (identity proofing) and AAL (authentication) are distinct assurance dimensions.

Source: NIST SP 800-63B-4 (2025), §2 Authentication Assurance Levels

Threats, Robustness and Usability

- **Prevent online guessing and credential stuffing**
  - Rate limits, breached-password screening and abuse monitoring.
- **Resist replay and phishing**
  - Fresh challenges; origin/RP binding where available.
- **Limit damage from database compromise**
  - Slow password hashing, unique salts and protected keys.
- **Make secure behaviour usable**
  - Password managers, accessible authenticators and reliable recovery.
- **Availability matters**
  - Avoid lockout-based denial of service; provide resilient services.

Entities and Deployment Models

Entities

- **People**
  - Local users, remote users, administrators
- **Machines**
  - Devices, hosts, workloads and APIs
- **Services**
  - Websites, SSH servers and identity providers

Deployment model

- **When?**
  - At connection, on step-up, and on reauthentication.
- **Which direction?**
  - One-way or mutual authentication.
- **What is checked?**
  - Account, device, service or a signed identity assertion.

Authentication Interaction Patterns

- **Credential presentation**
  - The claimant submits evidence, e.g., a password, over an authenticated encrypted channel.
  - The verifier compares it with stored verification material.

- **Challenge–response**
  - The verifier sends fresh data; the claimant computes a proof using a secret or private key.
  - A correct response alone does not guarantee phishing resistance.

Important properties

- Freshness, authenticated verifier, context binding and session binding.

Password Verification and Storage

- **A server must verify a password without storing it in plaintext.**
  - At registration: generate a unique random salt and compute a slow password hash.
  - At login: recompute with the stored salt and parameters; compare safely.
- **Modern choices: Argon2id; scrypt, bcrypt or PBKDF2 when appropriate.**
  - SHA-1/MD5, unsalted hashes and reversible encryption are unsuitable for password storage.
- **Always protect credential submission with authenticated TLS.**

Reference: OWASP Password Storage Cheat Sheet (Argon2id and salted password hashes).

Password Hashing: Modern Verification

Registration: derive a verifier, not a reversible password



Login: recompute with the stored salt and parameters; compare the derived hash safely.

Source: OWASP Password Storage Cheat Sheet. Historic DES crypt / MD5 shown in the original deck are obsolete.

Password Authentication: Threats and Defenses

- **Principal threats**
  - Phishing and malicious look-alike sites.
  - Credential reuse, spraying and credential stuffing.
  - Offline cracking after a password database breach.

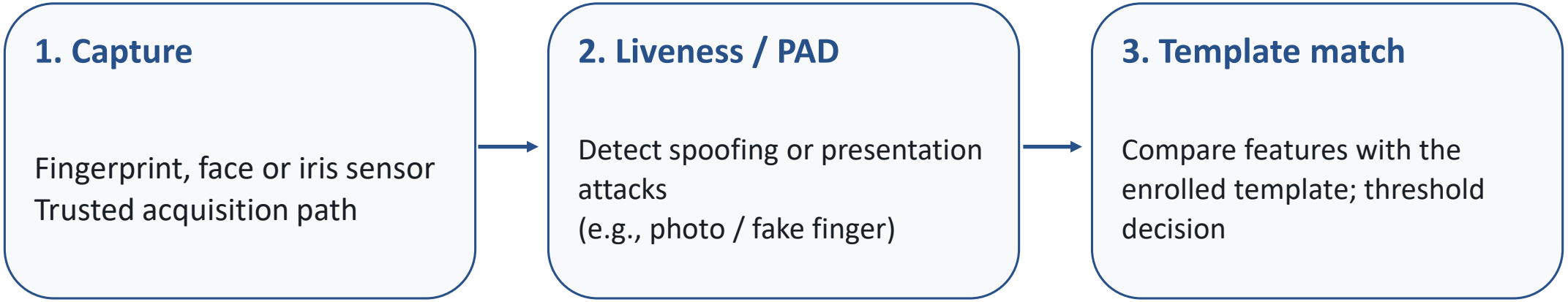
- **Recommended defenses**
  - Use long unique passwords and a password manager.
  - Screen against known compromised passwords; limit online attempts.
  - Protect login with TLS and add phishing-resistant MFA when possible.

Biometric Authentication: Enrolment and Matching

- **Enrolment**
  - Acquire a biometric sample and create a reference template.
- **Verification (1:1)**
  - Compare a new sample with the template for a claimed identity.
- **Identification (1:N)**
  - Search for a match across a reference population.
- **Matchers are probabilistic**
  - False accept/reject rates depend on a decision threshold.

Secure capture and presentation-attack detection (PAD) are important.

Biometric Authentication: Two Essential Checks



The biometric is a measurement—not a secret. Prefer local verification that unlocks a cryptographic key.

Biometrics: Advantages

- **No memorized secret is required for biometric matching.**
  - Convenient local device unlock and step-up verification.
- **Can activate a cryptographic authenticator.**
  - Example: fingerprint unlocks a passkey's signing operation.
- **Biometric verification may improve usability and accessibility.**
  - It must still provide alternatives and reliable recovery.

Biometrics: Security and Privacy Limits

- **Biometric traits cannot be rotated like passwords.**
  - Compromised templates require special protections and recovery.
- **Sensors and matchers can be fooled.**
  - Evaluate spoofing, PAD, false matches and demographic effects.
- **Privacy and accessibility matter.**
  - Minimize storage/sharing; provide alternatives and meaningful consent.

A remote service should normally receive cryptographic proof—not raw biometric samples.

One-Time Passwords (OTPs): Examples

- **A one-time password is valid for a limited use or time.**
  - Recovery / backup codes: pre-generated, individually consumed.
  - HOTP: generated from a shared secret and counter.
  - TOTP: generated from a shared secret and time step.

- **Examples: authenticator apps, physical OTP tokens, recovery-code lists.**

Recovery code	Authenticator app	Physical OTP token
F7K4–X9P2 One use, stored offline	482 719 Usually refreshes every 60 s	928 163 May use HOTP or TOTP

One-Time Passwords: Security Properties

- **Benefits**
  - A recorded code should not work again after successful use.
  - The user does not memorize a new secret for every sign-in.

- **Important limitations**
  - An attacker can relay a valid code to the real service in real time.
  - OTP entry is generally not phishing-resistant.
  - Use TLS, rate limiting, short validity and replay detection.
  - Backup codes must be stored securely and invalidated after use.

HOTP versus TOTP

- **HOTP — HMAC-based OTP (RFC 4226)**
  - Code = HOTP(K, C), using a shared secret K and moving counter C.
  - Client and server must resynchronize counters.

- **TOTP — Time-based OTP (RFC 6238)**
  - Code = HOTP(K, floor((T – T_0)/X)).
  - Typically X = 30 seconds; server tolerates small clock skew.

Both require protected seeds, throttling and replay checks.

HOTP (RFC 4226): From Counter to Code

- **Input: high-entropy shared key K and counter C.**
  - Compute HMAC-SHA-1(K, C) as specified by HOTP.
  - Dynamic truncation extracts a 31-bit integer D.
  - Display an integer code: D mod 10^d (often d = 6 or 8).
- **The verifier checks a limited counter window.**
  - Advance counters on success; track replay and throttle guesses.
- **Never treat the short decimal code as a high-entropy secret.**

TOTP (RFC 6238): From Time to Code

Time is mapped to a discrete moving counter:

$$C_t = \text{floor}((T - T_0) / X) \qquad \text{TOTP}(K) = \text{HOTP}(K, C_t)$$

0–29 s

482 719

30–59 s

906 142

60–89 s

573 095

Example: X = 30 s. Verifiers accept only a small time window and reject replay.

The Challenge–Response Pattern

1. **The verifier sends an unpredictable, fresh challenge.**
   - A random nonce prevents reuse of a prior response.
2. **The claimant computes a proof using a secret or private key.**
   - Example: HMAC for shared keys, signature for a private key.
3. **The verifier validates the proof and the context.**
   - Bind the exchange to the account, relying party and session.

Mutual authentication requires proof from both parties.

Challenge–Response: Benefits and Limitations

- **Strength: the private key or shared key need not be transmitted.**
  - Fresh challenges prevent simple replay of observed responses.

- **Caution: security depends on the credential and protocol.**
  - Low-entropy passwords can permit offline guessing.
  - A verifier storing a shared key becomes a compromise target.
  - Relayed challenges can still enable phishing without context binding.

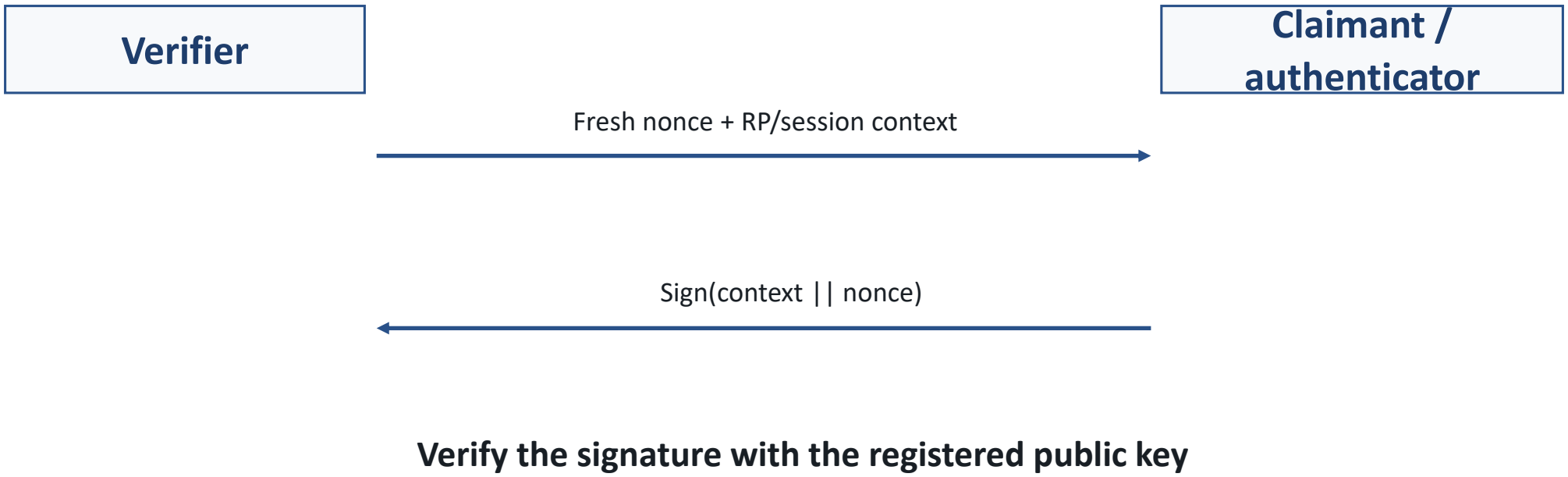
Prefer standardized, vetted protocols over custom constructions.

Authentication with Asymmetric Key Pairs

- **The claimant has a public/private key pair.**
  - The private key may be hardware-protected or stored in a secure element.
- **The verifier stores or validates the corresponding public key.**
- **A fresh, contextualized challenge is signed by the claimant.**
  - The verifier checks the signature and challenge freshness.
- **PIN, biometric or user presence may activate the key.**
  - A stolen public-key database does not reveal private keys.

Public-Key Challenge–Response

- **A signed response proves control of a private key.**
  - The signed material must include freshness and context.



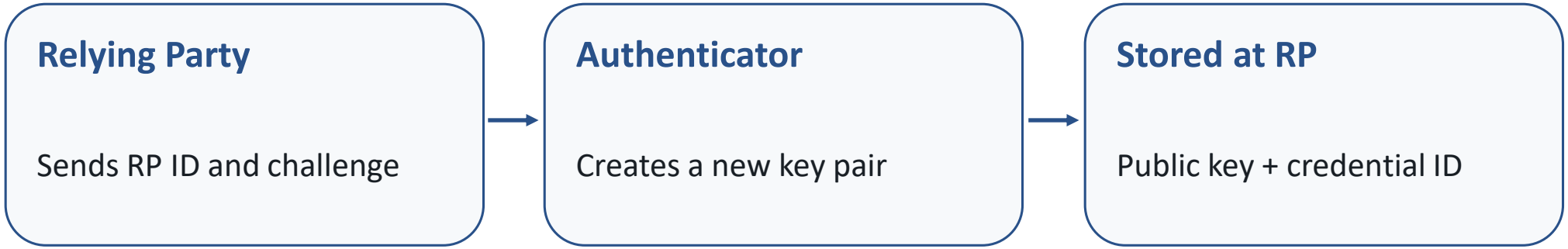
FIDO2, WebAuthn and Passkeys

- **FIDO2 combines WebAuthn (web API) and CTAP (Client To Authenticator Protocol).**
  - Each relying party gets a public-key credential scoped to its RP ID (RP == Relying Party).
- **Sign-in proves possession by signing a fresh challenge.**
  - The browser binds the ceremony to the legitimate web origin.
  - User presence and optional user verification (PIN/biometric).
- **Passkeys can be device-bound or securely synchronized.**
  - Phishing-resistant when origin/RP checks are correctly enforced.

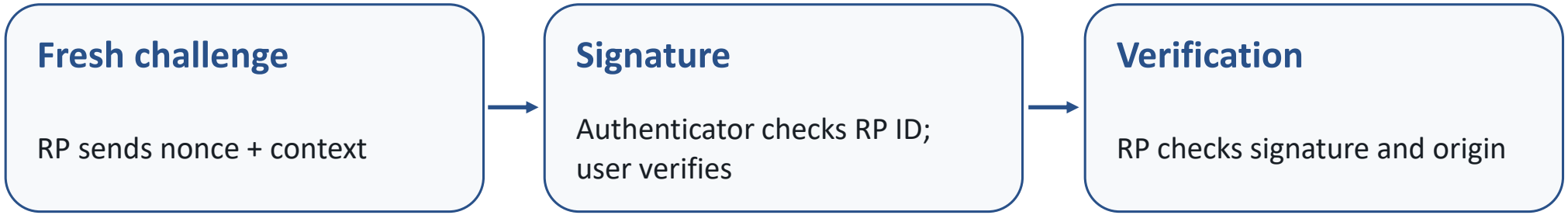
Reference: W3C WebAuthn; FIDO Alliance passkeys documentation.

Passkeys: Registration and Sign-In

REGISTRATION



SIGN-IN



FIDO2 uses a unique key per relying party; synced passkeys may synchronize private keys through an encrypted provider.

Challenge–Response with a Shared Secret

- **Verifier and claimant share a high-entropy symmetric key K.**
  - For a nonce N: response = HMAC(K, N || protocol context).
  - Verifier independently computes and compares the MAC.
- **Do not substitute a human password for a random key.**
  - Observed challenge/response pairs can enable offline password guessing.
- **Provision, rotate and protect shared keys on both sides.**

Password-Based Challenge–Response: Pitfalls

- **Naïve design: response = hash(password || challenge).**
  - An eavesdropper can test dictionary guesses offline.
  - Simple encryption of a challenge under a password has related risks.
- **Better designs**
  - Use a well-studied PAKE (e.g., OPAQUE) when password-based mutual authentication is required.
  - Or use a public-key authenticator, e.g. WebAuthn/passkeys.
- **Do not invent authentication protocols.**

PAP and CHAP: Legacy PPP Authentication

- **PAP: Password Authentication Protocol (PPP)**
  - Username and password are sent to the peer; no cryptographic protection inherent to PAP.
- **CHAP: Challenge-Handshake Authentication Protocol (PPP)**
  - Uses a challenge and an MD5-based response using a shared secret.
  - Provides one-way peer authentication and resists simple replay.
- **Legacy status**
  - Neither is appropriate as a modern Internet login protocol.
  - Avoid CHAP / MS-CHAP variants in new designs; prefer modern EAP/TLS methods.

Challenge–Response with a Shared Key

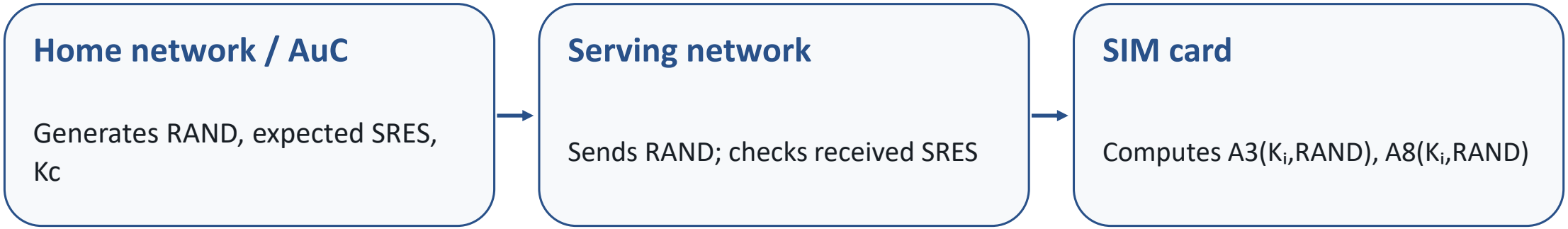
- **A randomly generated symmetric key is stronger than a human password.**
  - Verifier sends nonce N; claimant returns HMAC(K, N || context).
  - Verifier checks response with the same K.
- **Common in constrained and embedded systems.**
  - Benefits: simple and efficient; resists dictionary attacks with random K.
  - Costs: secure provisioning, storage, rotation and compromise recovery.

GSM Subscriber Authentication (Legacy 2G)

- **GSM / 2G SIM and home network share a subscriber key K_i.**
  - The SIM never sends K_i over the radio interface.
- **The home network supplies challenge-response triplets.**
  - The SIM computes SRES and session key Kc from RAND and K_i.
- **Historical limitations**
  - GSM authenticates the subscriber but not the network to the subscriber.
  - This enables rogue base-station risks; 3G/4G/5G use different AKA protocols.

GSM: Authentication Triplets (Legacy 2G)

- **Home/AuC computes from K_i and RAND:**
  - RAND (128 bits), SRES (32 bits), Kc (64 bits).
- **Serving network verifies the SIM-computed SRES.**
  - Subscriber key K_i is never broadcast.



Authentication of Hosts and Devices

- **A hostname, IP address or MAC address alone is not proof of identity.**
  - Spoofing or redirection can invalidate trust based only on identifiers.
- **Cryptographic machine identities**
  - Symmetric keys, SSH host keys or certificate-backed public keys.
  - A trusted binding connects the key to the intended host/service.
- **For TLS, verify the certificate chain and expected DNS name.**
  - DNSSEC authenticates DNS data; it is not by itself server authentication.

Authentication of Services

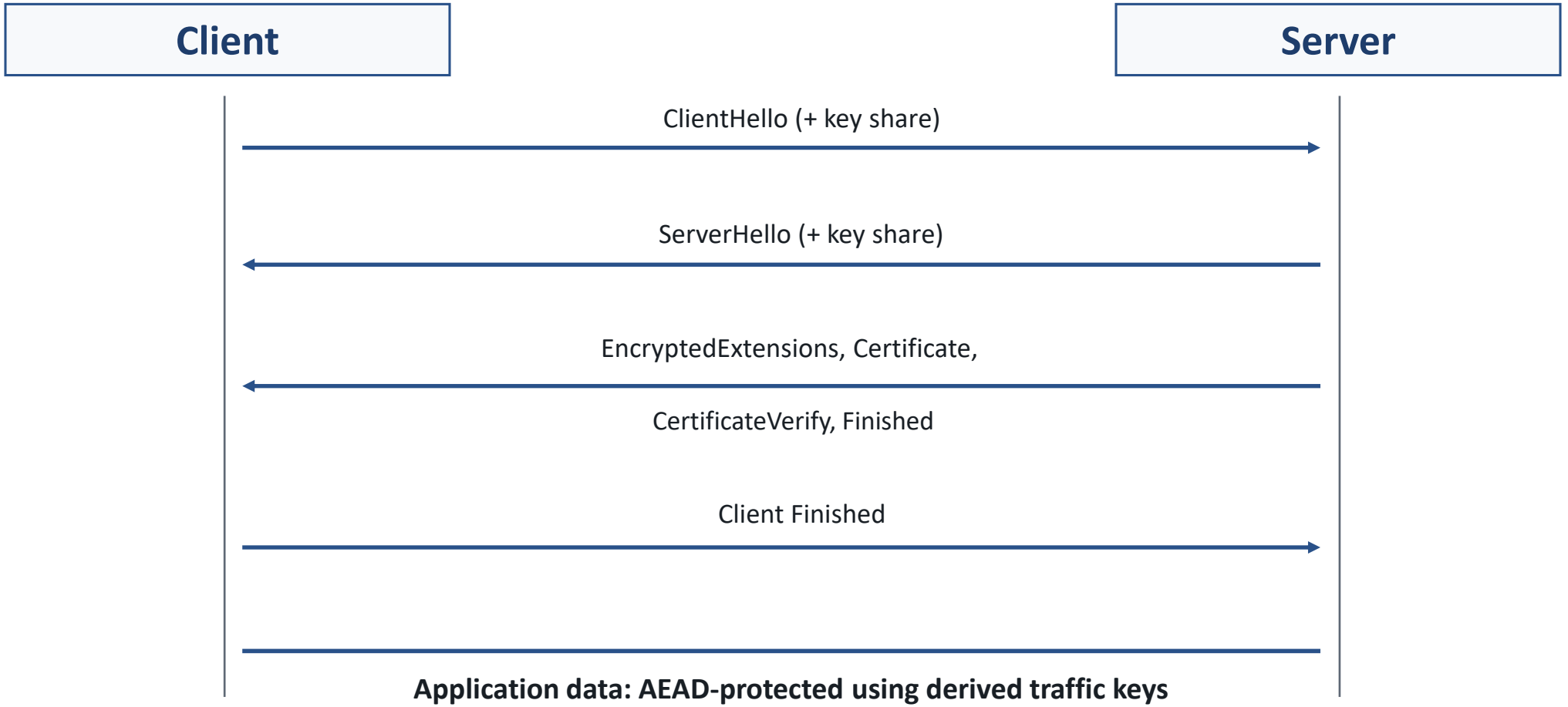
- **Authenticate the intended service endpoint—not merely its IP address.**
  - In HTTPS, verify that the TLS certificate matches the requested name.
- **Service identities can use distinct keys and certificates.**
  - Mutual TLS also authenticates clients when configured.
- **Application-layer authentication may still be needed.**
  - API keys, signed tokens and OAuth access tokens are different credentials.
- **A shared host does not automatically confer a shared service identity.**

TLS 1.3: Authentication in a Secure Session

- **TLS 1.3 (RFC 8446) protects application traffic over TCP.**
  - Confidentiality, integrity and endpoint authentication.
- **An ephemeral (EC)DHE exchange derives fresh traffic keys.**
  - Server proves certificate-key ownership via CertificateVerify.
  - Client checks CA trust chain and expected server name.
- **Client certificate authentication is optional in TLS.**
  - User login is normally implemented above TLS.

Reference: RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3.

TLS 1.3: Simplified Server Authentication



Simplified full TLS 1.3 handshake (RFC 8446). The server certificate and proof are authenticated within the handshake.

Cipher Suites in TLS 1.3

- **TLS 1.3 negotiates independently:**
  - Cipher suite: AEAD encryption plus HKDF hash.
  - Key exchange group: e.g. X25519 / P-256.
  - Signature algorithms: e.g. ECDSA / RSA-PSS / Ed25519.

- **Example: TLS_AES_128_GCM_SHA256**
  - AEAD: AES-128-GCM; HKDF hash: SHA-256.

Unlike TLS 1.2, the cipher suite does not encode key exchange or authentication algorithms.

Reference: RFC 8446, §§4.1–4.2 — TLS 1.3 cipher suite and separate negotiation.

SSH: Secure Remote Access

- **SSH provides protected remote access and transport.**
  - Interactive shell, remote commands, SFTP and port forwarding.
- **Handshake establishes encrypted, integrity-protected keys.**
  - Server proves possession of its host private key.
- **User authentication follows over the protected channel.**
  - Password, public-key signature, hardware keys or configured methods.

SSH and TLS authenticate entities differently.

SSH: Host and User Authentication

- **Host authentication**
  - Client checks a pinned known_hosts key, trusted host CA or other trusted binding.
  - First-use trust (TOFU) needs out-of-band verification for strong assurance.
  - An unexpected host-key change is a security warning.

- **User authentication**
  - Password or signature with the user's private key.
  - The server checks the corresponding authorized public key.
  - Keep user keys protected; prefer hardware-backed keys where suitable.

Centralized Network Authentication (AAA)

- **AAA = Authentication, Authorization and Accounting.**
  - Access device / NAS forwards identity exchanges to a central AAA server.
  - RADIUS and Diameter support network access workflows.

- **802.1X + EAP is common in enterprise wired and Wi-Fi networks.**
  - EAP-TLS can authenticate devices/users with certificates.

Protect AAA transport and shared secrets; do not assume all RADIUS deployments are encrypted.

Federated Authentication with an IdP

- **An Identity Provider (IdP) authenticates the user.**
  - A Relying Party (RP) accepts a verified identity assertion.
- **Federation allows several services to trust one IdP.**
  - Assertions/tokens contain specific claims, not unrestricted identity data.
- **Examples of protocols: OpenID Connect and SAML.**
  - The RP checks issuer, audience, signature, lifetime and context.

User consent and attribute minimization protect privacy.

Centralized Authentication: Trade-Offs

- **Benefits**
  - Consistent policy, MFA, account lifecycle and monitoring.
  - Fewer services need to store user passwords.
  - SSO can reduce repeated logins.

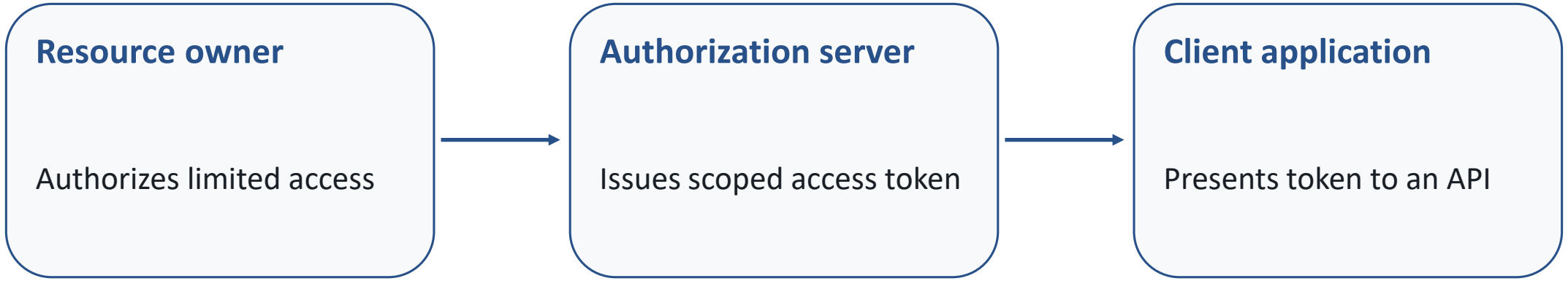
- **Risks and design considerations**
  - IdP compromise can affect many relying parties.
  - Outage can block access; plan resilience and emergency accounts.
  - Centralized login creates privacy and correlation considerations.

Single Sign-On (SSO)

- **SSO reuses an existing authentication session across services.**
  - User authenticates with an IdP; later RPs receive fresh assertions/tokens.
- **An RP still must validate each assertion and apply authorization.**
  - SSO is a user-experience property, not a new authentication factor.
- **Security controls**
  - Session timeouts, step-up for sensitive actions, logout and revocation.
  - SSO reduces password entry but raises session-token protection requirements.

OAuth 2.0: Delegated Authorization

- **OAuth 2.0 lets a client access an API with delegated permissions—not the user's password.**



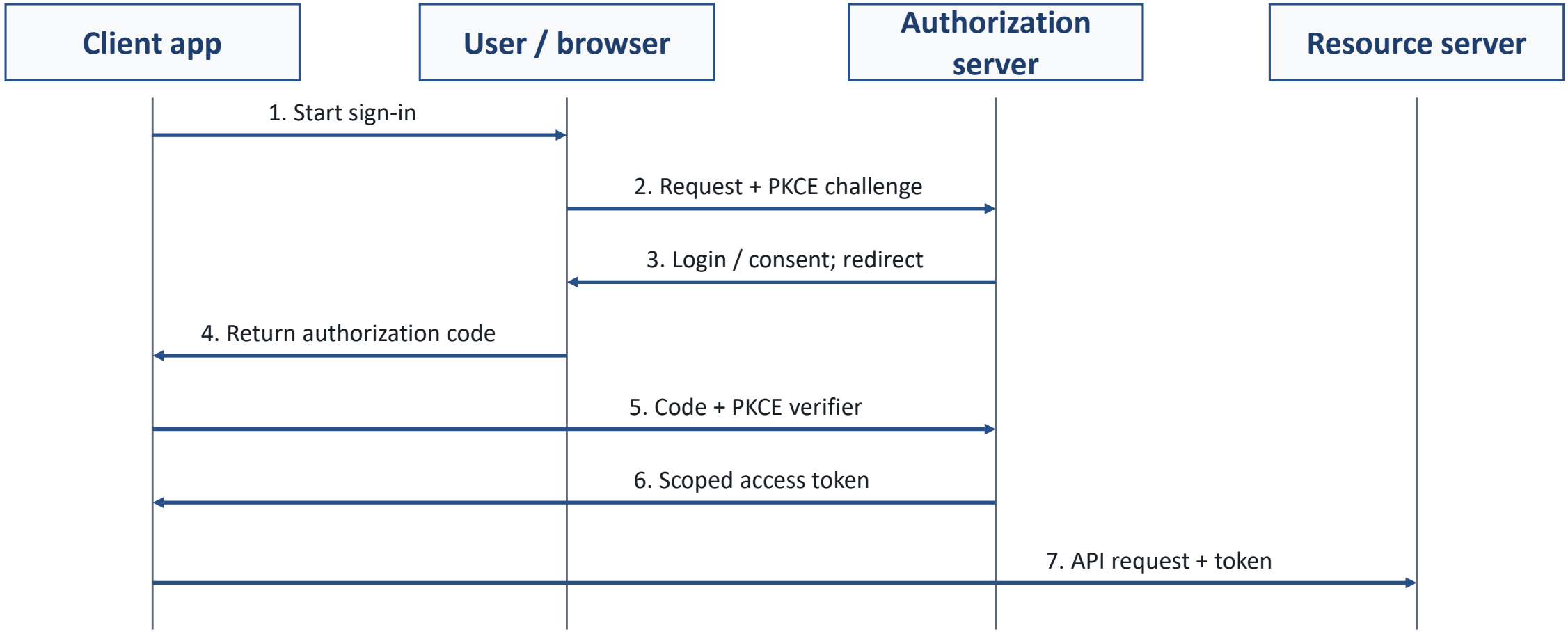
Resource server validates token audience, scope and expiry before serving data.

Key idea: OAuth access tokens represent permission to access resources; they are not identity tokens.

OAuth 2.0: Roles and Tokens

- **Resource owner**
  - Entity that can authorize access to protected resources.
- **Resource server (RS)**
  - API that validates an access token and serves the requested resource.
- **Authorization server (AS)**
  - Issues access tokens after authorization checks.
- **Client**
  - Application requesting API access, possibly on a user's behalf.
- **Access token**
  - Represents delegated permissions; constrain scope, audience and lifetime.
- **Crucial distinction**
  - OAuth 2.0 is for authorization—not proof of user identity to a client.

OAuth 2.0: Authorization Code with PKCE



RFC 9700: PKCE is required for public clients; authorization codes are exchanged at the token endpoint.

OpenID Connect (OIDC): User Authentication

- **OIDC adds an ID token (an authenticated identity assertion) to OAuth 2.0.**
  - Relying party validates its issuer, audience, signature, expiry and nonce as appropriate.

ID token

Audience: client / RP
Purpose: who signed in?
Issuer + subject + claims

Access token

Audience: API / resource server
Purpose: what may be accessed?
Scopes and other restrictions

OIDC Core 1.0 · Do not use an OAuth access token as a substitute for an OIDC ID token.

Bibliografia / Recursos adicionais

Leitura recomendada:

- Security in Computing, 4th edition, C. P. Pfleeger, S. L. Pfleeger: Sec. 4.5
- Segurança em Redes Informáticas, A. Zúquete: Secs. 5.3, 5.4.1, 8.6.3, 8.9.2, Chap. 10, Sec. 11.3

Leitura adicional:

- Authentication
- Biometric authentication
- Dictionary attacks
- HMAC-based One-Time Password (TOTP)
- Time-based One-Time Password (TOTP)
- Yubikey
- GSM security
- SSH
- TLS (Transport Layer Security)
- AAA (Authentication, Authorization and Accounting)
- Federated identity
- Single Sign-On (SSO)
- OAuth 2.0
- What is OpenID Connect (OIDC)?
- OpenID Connect Playground